CSE 318 — Online 4

Section A2

Here, you will update your neighborhood generation: take a small consecutive segment (say 3 cities) and reinsert it elsewhere *in the same order* (no reversal). It often escapes different kinds of local optima.

What to implement.
Write `generateNeighborOrOpt(tour)` that returns one neighbor tour. Note that city 0 is fixed at position 0 and never participates in a move.

1. Pick a segment length L uniformly at random from $\{3, 4, 5\}$.
2. Pick a random start index i with $1 \leq i$ and $i + L - 1 \leq N - 1$
3. Remove the sub-list `seg = tour[i ...i+L-1]` from the tour.
4. Pick a random insertion index j with $1 \leq j \leq$ length after removal, and $j \neq i$ (do not put it back exactly where it was).
5. Insert `seg` at position j, preserving order.

Plug this function in place of your current neighbor generator. Keep everything else in SA identical (initial solution, temperature, cooling, acceptance rule).

Output:

- Run modified SA **5 times** per instance, on **at least two** test instances. Report **best, average, worst** cost.
- Compare directly against your original SA with the same parameters (T_0, α, iteration budget). Report best/avg/worst for both.